# COMP 3920 Database Systems 1

# MySQL
# Users and Roles

Introduction:
The goal of this document is to serve as a walkthrough creating some users and roles into MS SQL Server.

We are going to be using a sample database for a Classic Model Store.
For more information: https://github.com/hhorak/mysql-sample-db

We will be creating 8 users and 4 roles as such:

1. Helen and Henry are in **HR**.
2. Ian and Irene are in **Inventory**.
3. Sally and Sam are in **Sales**.
4. Mary and Mark are in **Management**.

Step 1:
Download `classicmodels MS SQL.zip` and unzip.

Step 2:
Use the MS SQL `sa` account to import the data from inside `classicmodels MS SQL.sql`.

Confirm that the database was imported.

You should have the following tables with the following number of rows:

- dbo.customers
- dbo.employees
- dbo.offices
- dbo.orderdetails
- dbo.orders
- dbo.payments
- dbo.productlines
- dbo.products

```sql
SELECT count(*) AS 'Number of Customers' FROM customers;
SELECT count(*) AS 'Number of Employees' FROM employees;
SELECT count(*) AS 'Number of Offices' FROM offices;
SELECT count(*) AS 'Number of Order Details' FROM orderdetails;
SELECT count(*) AS 'Number of Orders' FROM orders;
SELECT count(*) AS 'Number of payments' FROM payments;
SELECT count(*) AS 'Number of Product Lines' FROM productlines;
SELECT count(*) AS 'Number of Products' FROM products;
```

100 %

Results | Messages

| | Number of Customers |
|---|---|
| 1 | 122 |

| | Number of Employees |
|---|---|
| 1 | 23 |

| | Number of Offices |
|---|---|
| 1 | 7 |

| | Number of Order Details |
|---|---|
| 1 | 2996 |

| | Number of Orders |
|---|---|
| 1 | 326 |

| | Number of payments |
|---|---|
| 1 | 273 |

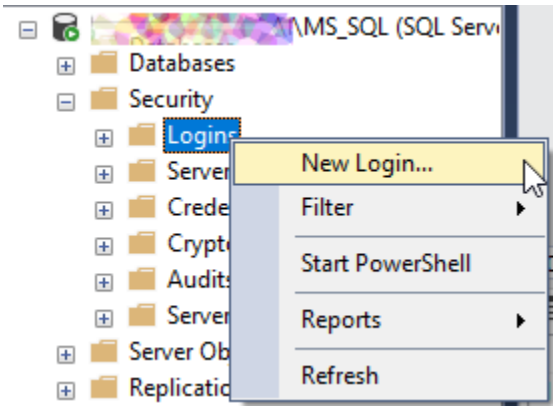
| | Number of Product Lines |
|---|---|
| 1 | 7 |

| | Number of Products |
|---|---|
| 1 | 110 |

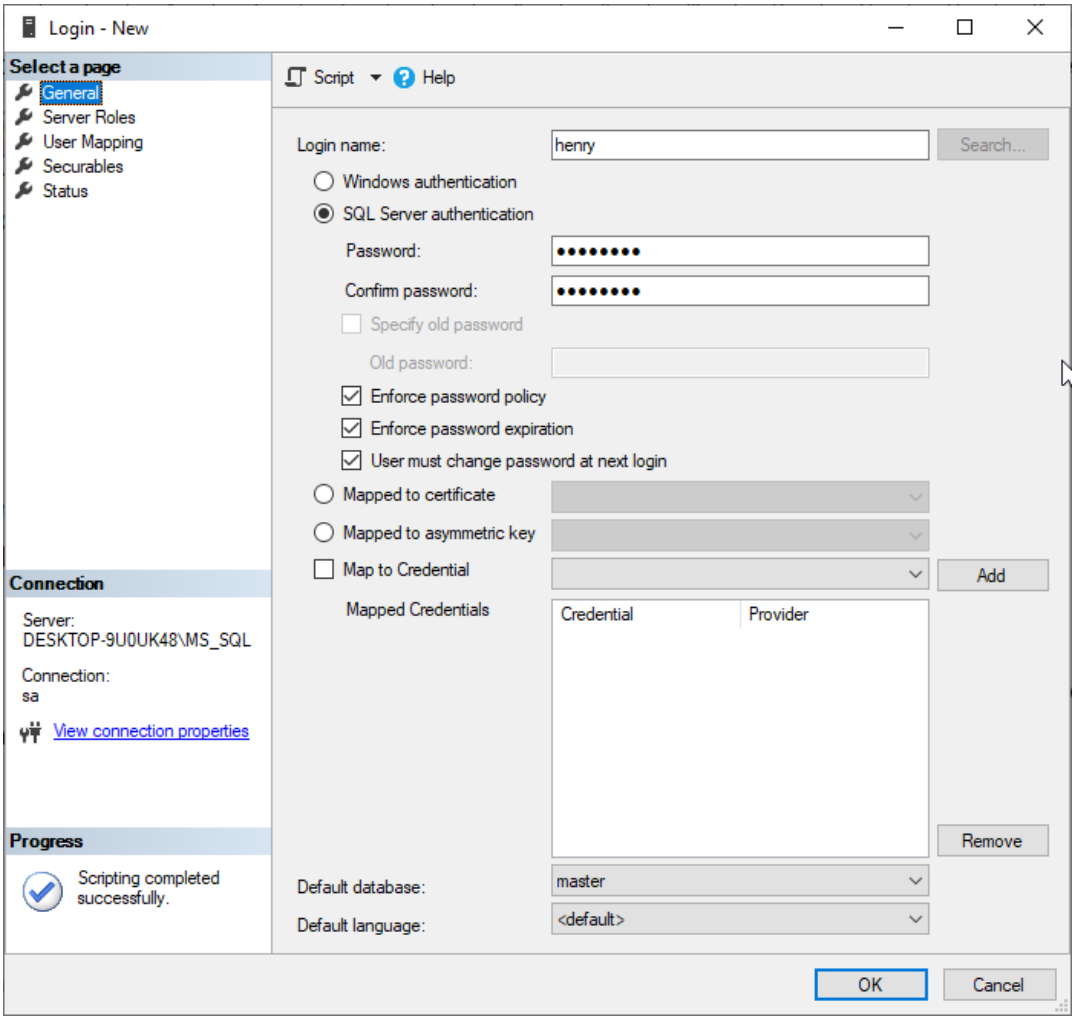
Step 3:
Create the 8 logins:
Henry, Helen, Ian, Irene, Mark, Mary, Sally, and Sam.

For simplicity, provide each user with the password: `Password`
Note that in a production environment, we would want to provide each user a unique and strong password (ex: greater than 10 characters, including symbols, uppercase, lowercase and numbers).

You can use the SSMS wizard to create logins by right clicking the Logins folder at the **Server level**.



When prompted in the wizard, provide the username, password and confirm the password




You can click on the script button Script to show what code SSMS will run behind the scenes when you click "OK".

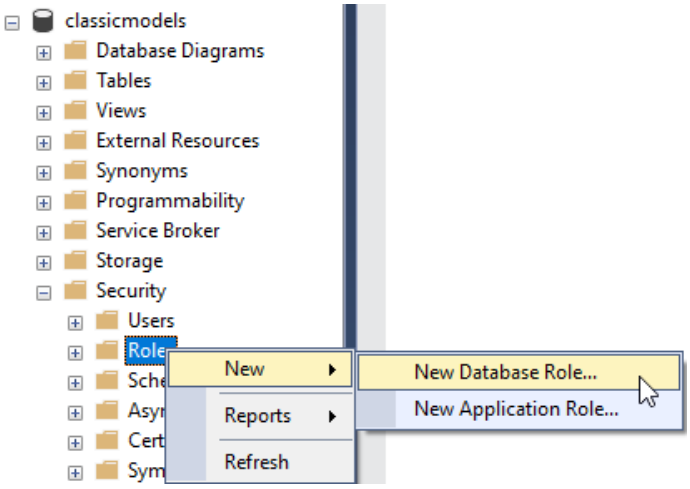
```
USE [master]
GO
CREATE LOGIN [henry] WITH PASSWORD=N'Password' MUST_CHANGE, DEFAULT_DATABASE=[master],
CHECK_EXPIRATION=ON, CHECK_POLICY=ON
GO
```

What it provides is a simple CREATE LOGIN command with the password and a few extra options like MUST_CHANGE, etc.

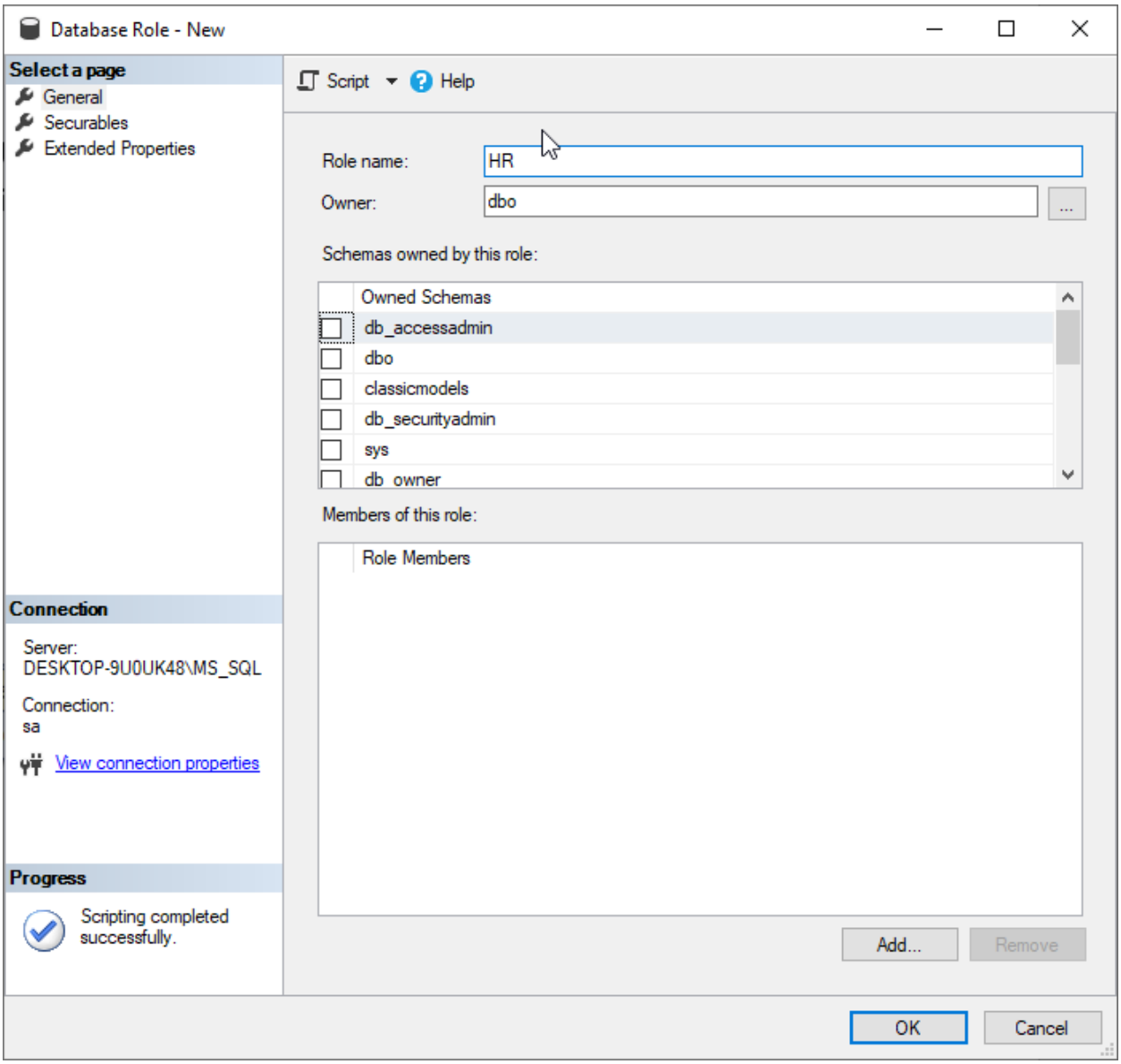
Note: Unlike MySQL, **MS SQL Server is NOT case sensitive** with Logins, Users and Role names.
It does not matter if you use uppercase or lowercase for the logins.

Step 4:
Create the 4 roles:
HR, Inventory, Sales and Management.

Within SSMS, expand the `classicmodels` database, expand "Security" and right click on "Roles" and click "New Database Role…" to open the roles wizard.



In the wizard, provide the name of the new role and `dbo` as the owner. (`dbo` is the default owner).



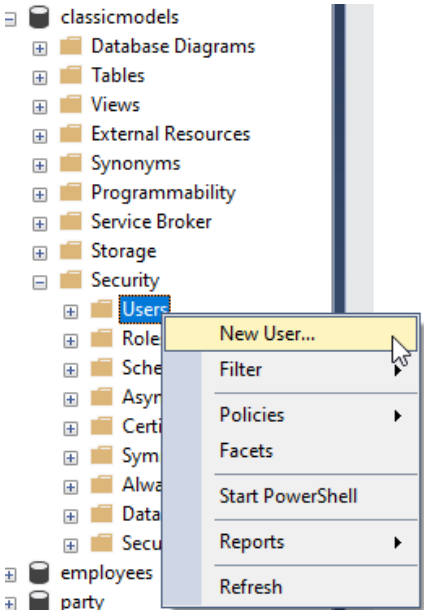

Click on "Script" to see the SQL commands behind the scenes:

```
USE [classicmodels]
GO
CREATE ROLE [HR] AUTHORIZATION [dbo]
GO
```

The wizard can also let you create and associate role members, but since we don't have any users yet, we'll leave this until later.

Step 5:
Create the 8 users in the `classicmodels` database:
Henry, Helen, Ian, Irene, Mark, Mary, Sally, and Sam.

Within SSMS, expand the `classicmodels` database, expand "Security" and right click on "Users" and click "New User…" to open the users wizard.

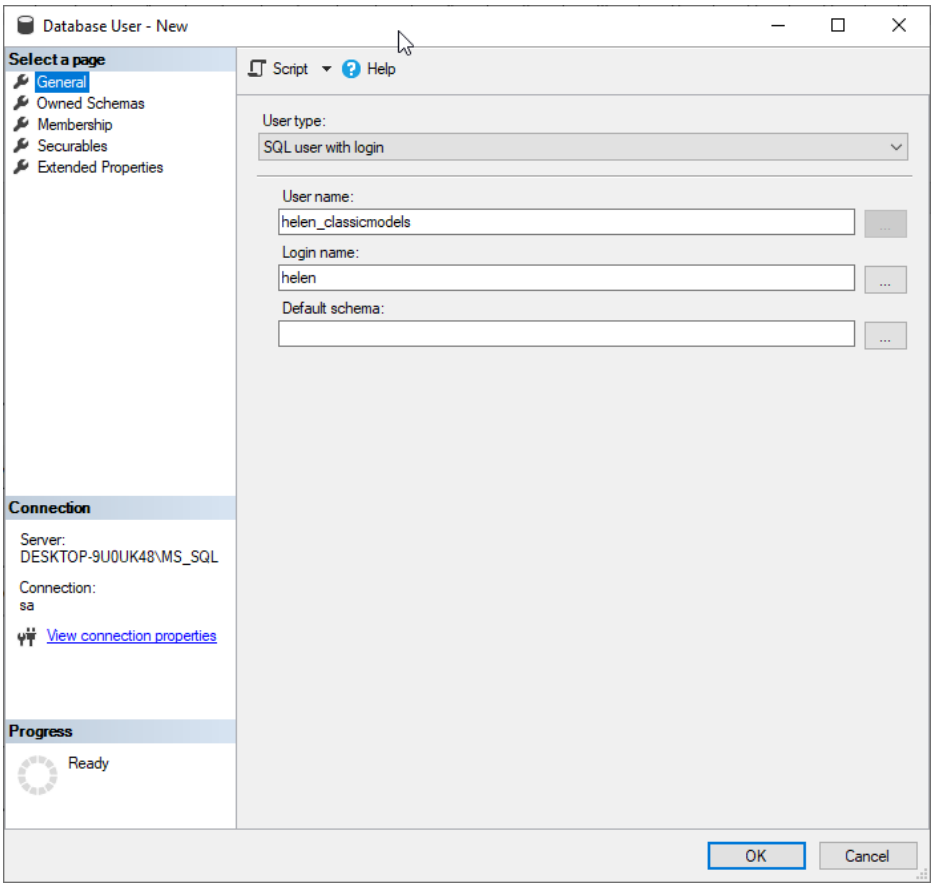



Users, in MS SQL are the link between logins (authentication with password) and particular databases that should have access to. Because of this, we can use the naming convention of:
**Login** + "_" + **DatabaseName**

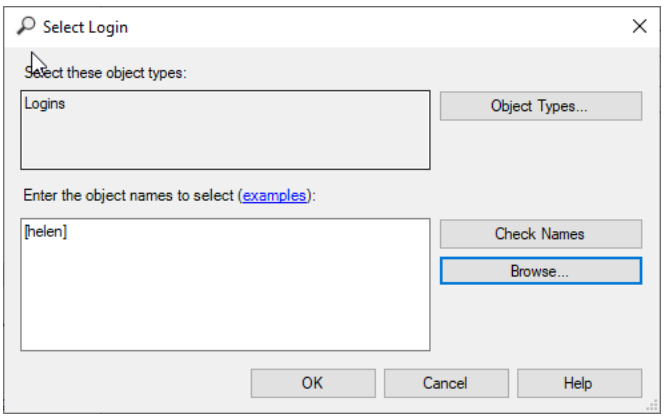
Ex: if we are creating **Helen** in the **classicmodels** database, our user would be:
**helen_classicmodels**

Note: Users don't *need* to be named using this convention (login + "_" + database) but can help us understand, in this lab, that users in MS SQL link the login to the database.

Specify the user name, then click on the three dots ... next to login to specify the logins you created in the earlier step.

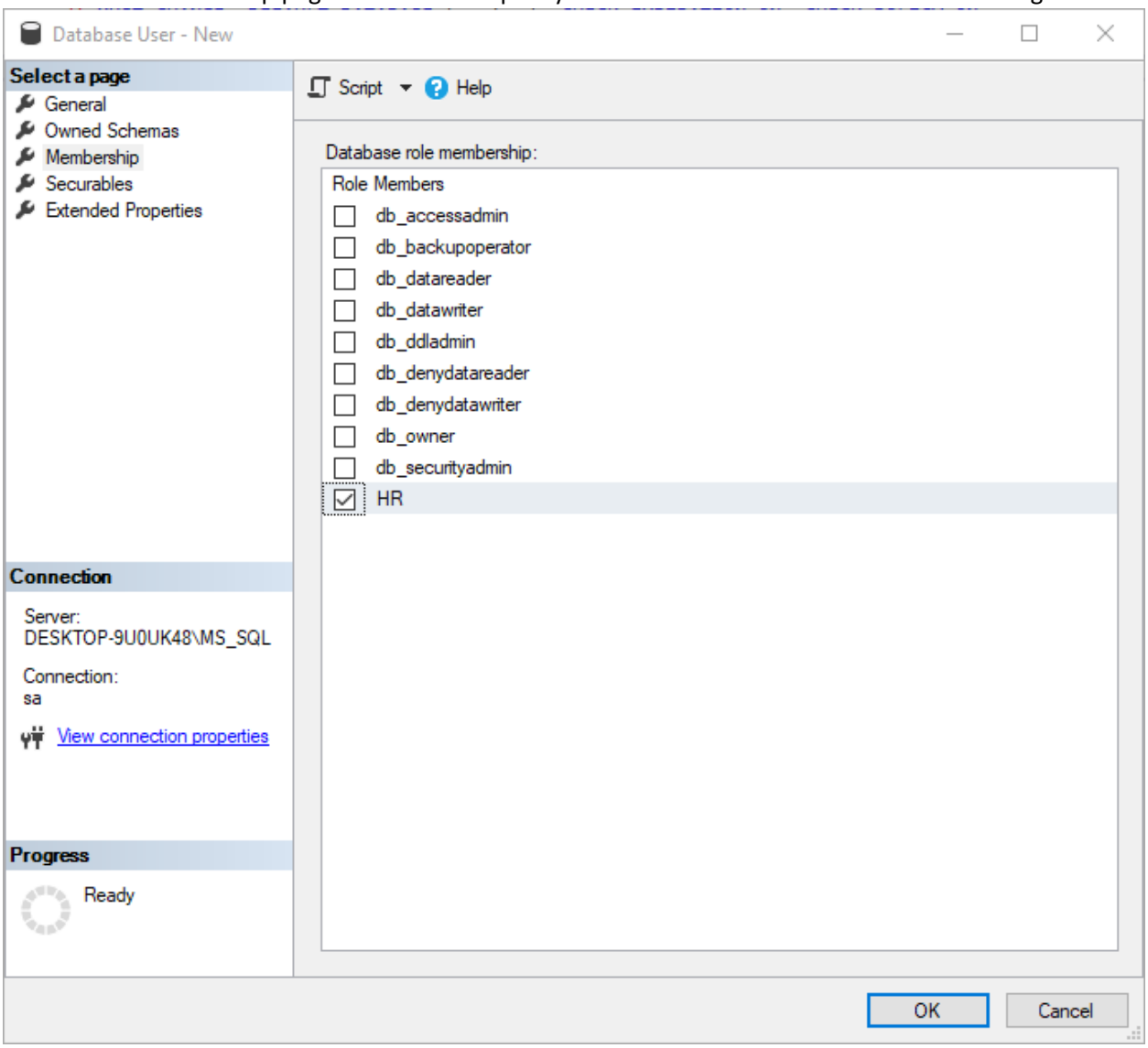



Click Browse to find the appropriate login.



This wizard will also let us add the user to the appropriate role, now that all the roles have been created.

Click on the Membership page on the left to specify which Role each of the users should belong to.



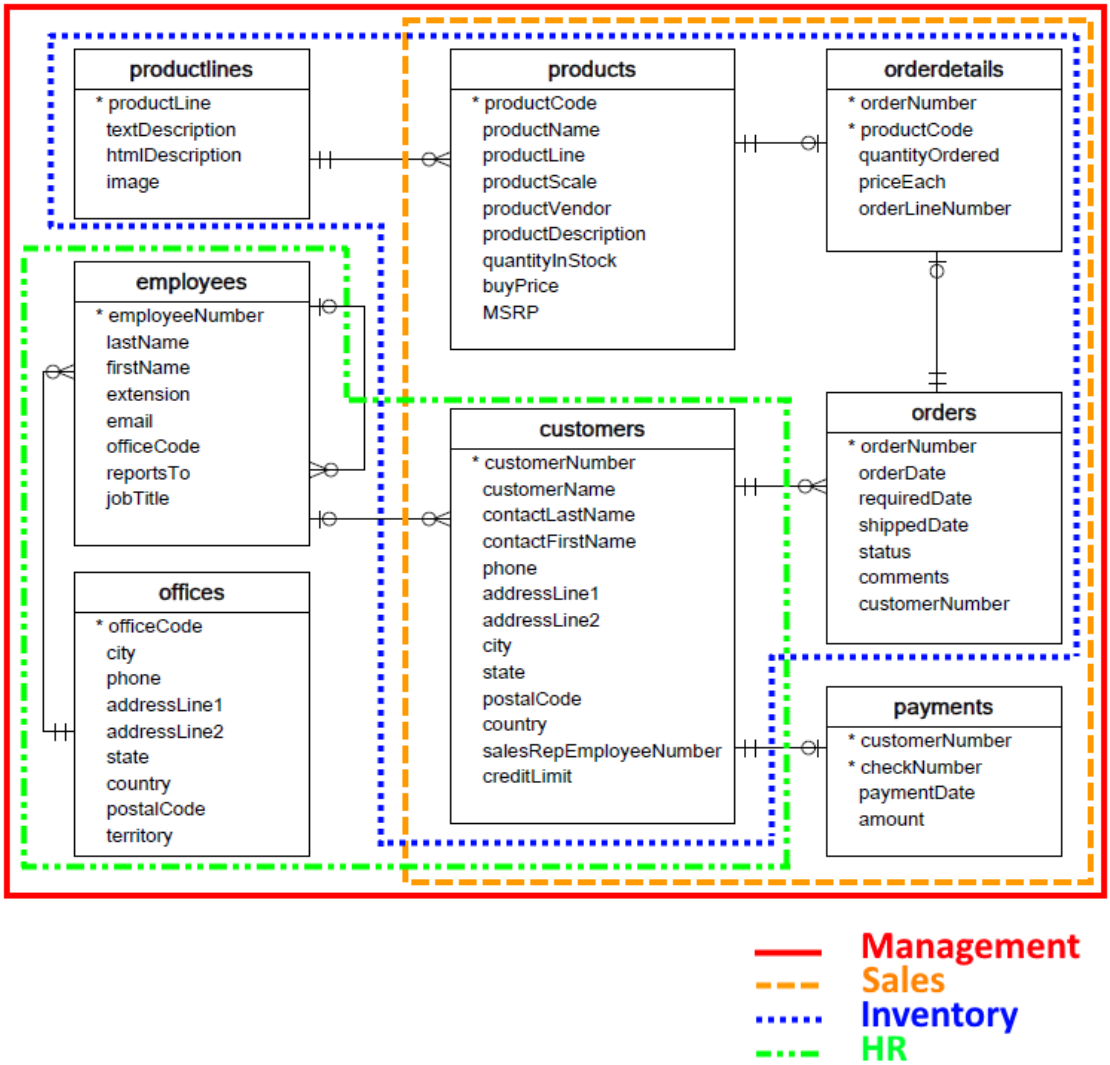

The "Script" button shows us that we will run a CREATE USER command that links the helen_classicmodels user to the helen login in the classicmodels database. It also runs an ALTER ROLE command to add helen_classicmodels to the HR role.

```
USE [classicmodels]
GO
CREATE USER [helen_classicmodels] FOR LOGIN [helen]
GO
USE [classicmodels]
GO
ALTER ROLE [HR] ADD MEMBER [helen_classicmodels]
GO
```

Assign each of our 8 users their appropriate roles:

1. Helen and Henry are in **HR**.
2. Ian and Irene are in **Inventory**.
3. Sally and Sam are in **Sales**.
4. Mary and Mark are in **Management**.

Step 6:
Grant the following privileges to each role as shown in the diagram below:
Privileges to grant: SELECT, INSERT, UPDATE, DELETE.

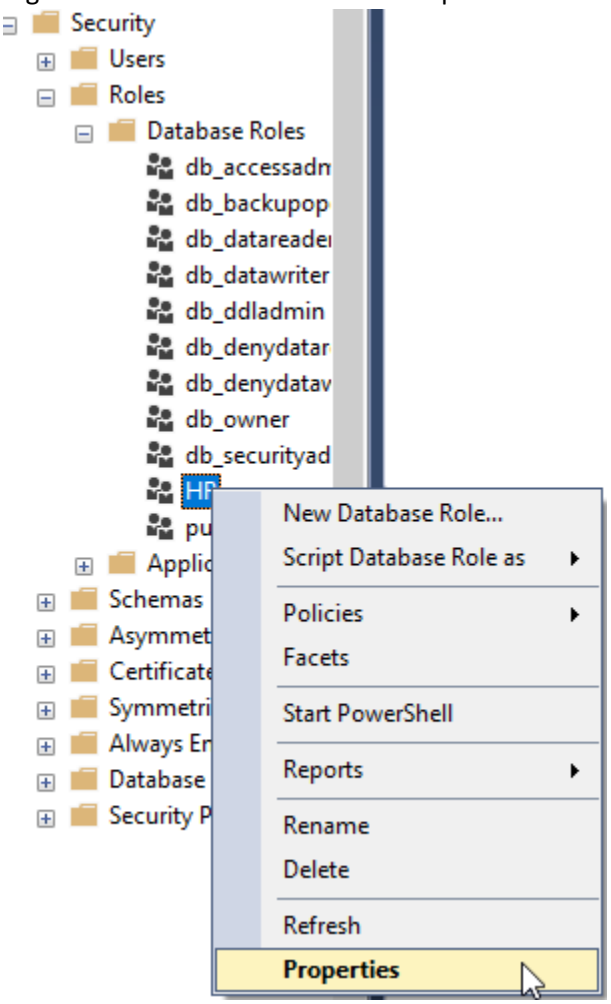



Ex: For the `HR` role we need to grant `SELECT`, `INSERT`, `UPDATE` and `DELETE` on the `employees`, `customers` and `offices` tables.

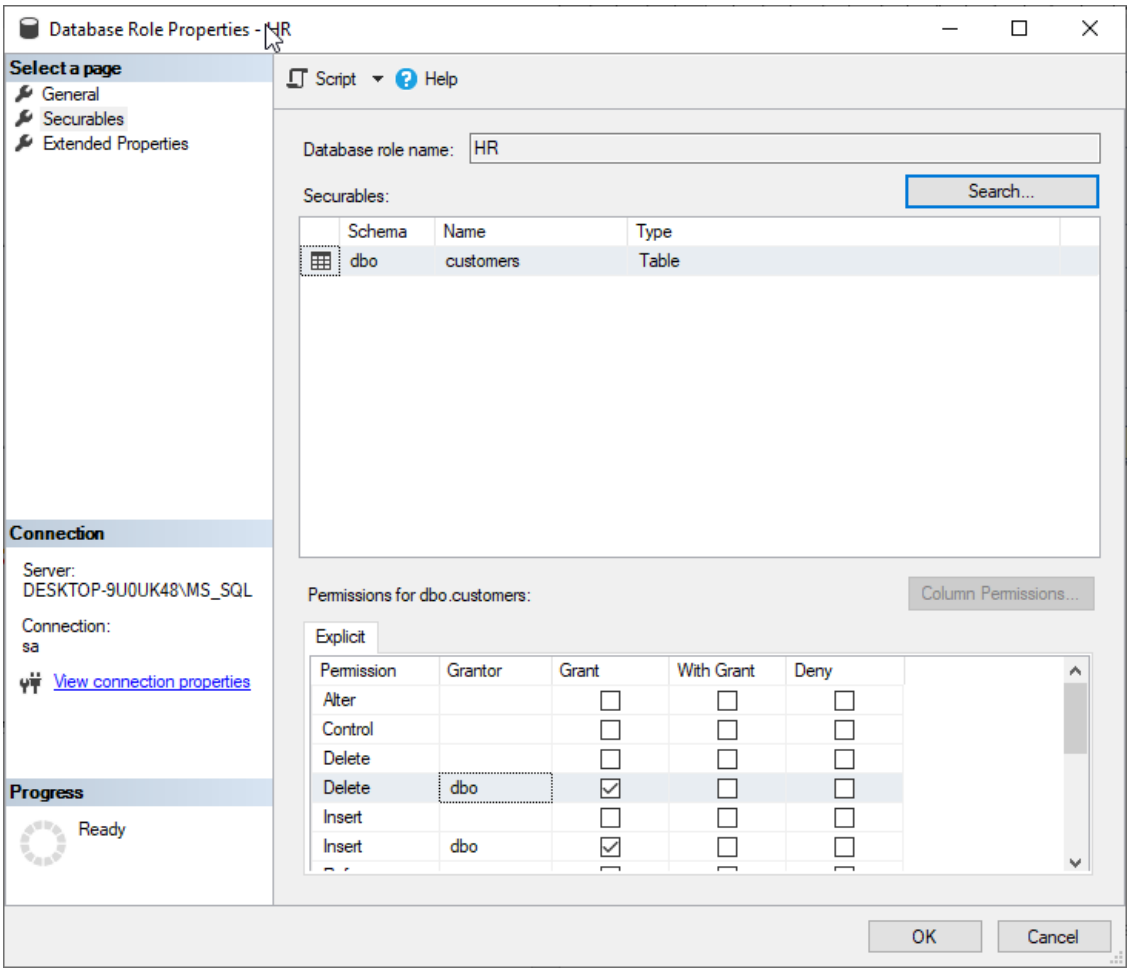
To grant privileges on the table to the roles there is, of course, another wizard in SSMS for this.

Now that you have all the roles created, you can add the privileges for them to the tables.

Right click on the roles and click "Properties":

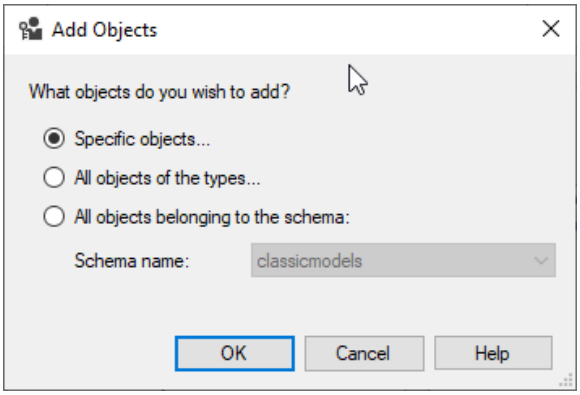


On the 2nd page, "Securables", you'll be able to "Search…" for your database tables and add them to the list.

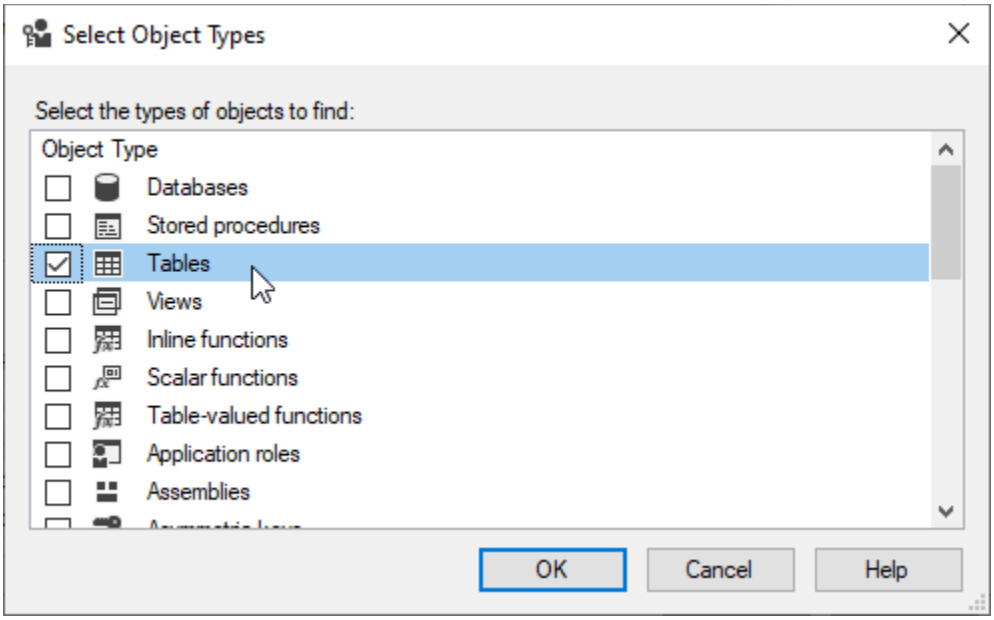



Once each of the tables are in the securables list, you can add the appropriate privileges (SELECT, INSERT, UPDATE, DELETE) in the permissions section.

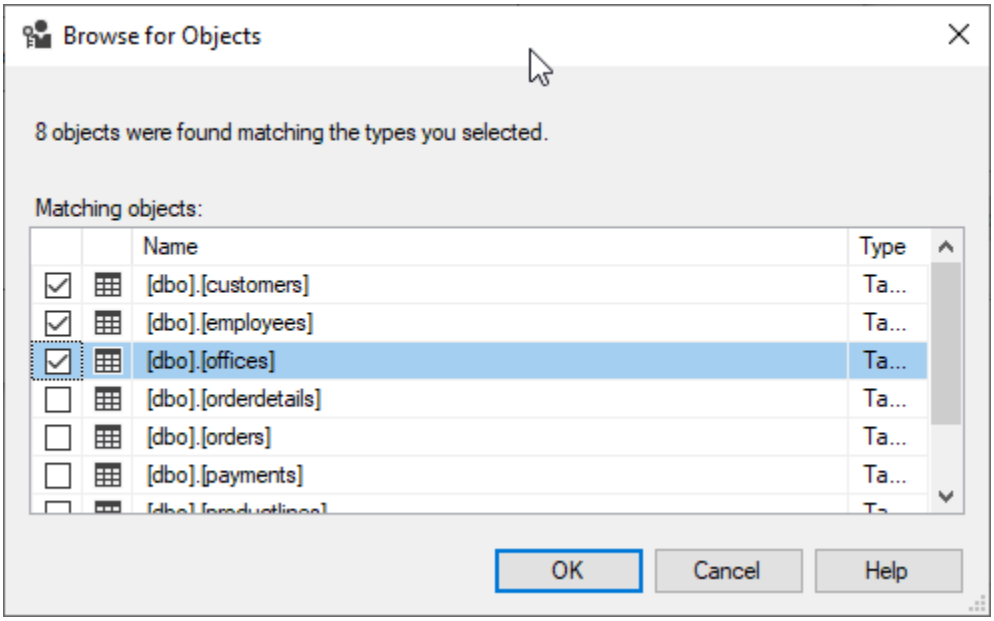
To add securables, click "Search" and look for "Specific Objects"



Select "Tables" as your Object Type and click "OK".




Click "Browse" to look for tables within your database.




Select all the tables that apply for this role and click "OK".

Once all the tables are added to the securables list, make sure they all have SELECT, INSERT, UPDATE and DELETE privileges.

The Script button should show the following changes to the permissions by creating GRANT statements (one for each of the SELECT, INSERT, UPDATE and DELETE).

```
use [classicmodels]
GO
GRANT DELETE ON [dbo].[customers] TO [HR]
GO
use [classicmodels]
GO
GRANT INSERT ON [dbo].[customers] TO [HR]
GO
use [classicmodels]
GO
GRANT SELECT ON [dbo].[customers] TO [HR]
GO
use [classicmodels]
GO
GRANT UPDATE ON [dbo].[customers] TO [HR]
GO
```

You can also combine each of these statements into a single one,

Re-open the roles wizard for each of the roles and add the appropriate privileges.

Open and execute the "Verify Users and Roles MS SQL.sql" file:

Save the results as text into a file called `02_Results_MS_SQL.rpt`

Submit your results file:
`02_Results_MS_SQL.rpt`

Submit

The initial contents of `02_Results_MS_SQL.rpt` should look like this:

```
login
-----------------------------------
##MS_PolicyEventProcessingLogin##
##MS_PolicyTsqlExecutionLogin##
helen
henry
```